Christian Bucher - General Technical Portfolio

General profile, capability map and evidence route

Portfolio thesis

My work spans agent infrastructure, memory and voice systems, AMR AI architecture, project planning, IP-style formalization and adversarial research boundaries.

How to read this portfolio

Start with the AI systems, then AMR/ECSA/FFG, then IP formalization, then Quantum/EXON as a red-team maturity case.

Remote WhatsApp-to-Agent Command Loop

A phone should be able to issue work to a home computer without turning the system into an unsafe direct shell. The design had to translate casual WhatsApp messages or audio into structured work orders, run them through a tool-enabled agent side, judge completion, and only then report back.

- This is the clearest Secure AI signal: remote natural-language instructions became structured work orders, then passed through tool execution and evaluation before a user-facing reply.
- Prototype/evolved system. Some EXOVIUM pieces were later superseded by Miguel Core. Private channel identifiers and credentials are excluded.

What I had in mind

What I had in mind was simple but technically demanding: I wanted to be away from my laptop and still delegate meaningful work to it without turning my phone into a blind remote shell. The interesting part was the trust boundary. A message had to become a work order, the work order had to be executed by the right side of the system, and the result had to survive evaluation before coming back to me.

What I built

- Implemented across OpenClaw watcher, MiguelAgent, EXOVIUM core, WA task manager, cognition routing, and architecture notes. Source notes document immediate ACK behavior, quality-loop thresholding, model routing, audio transcription, and two successful end-to-end tests.

Miguel Core Unified Daemon

Early systems had separate daemons, command scripts, and services. Miguel Core consolidated the control surface into a local Python daemon that could coordinate chat, streaming, voice, memory, model tiers, orchestration, state, and tool endpoints.

- This shows system-level building rather than model use: interfaces, state, memory, tools, routing, health, orchestration, and operating boundaries were treated as one control plane.
- Local experimental control plane, not a hardened commercial security product.

What I had in mind

Miguel Core came from a very practical frustration: I had too many strong pieces living as separate scripts and services. I wanted one local control layer that could make the system observable, routable and easier to reason about.

What I built

- Central file `miguel_core.py` documents the consolidation goal and exposes the route surface. `orchestra.py` implements async worker delegation to Opus, Codex, GPT-style workers through a subprocess layer with timeouts, cancellation, status, and stats.

Multi-Model Cascade Router

Agent systems fail when all tasks are pushed through one model. The router treats models as resources with strengths, costs, quotas, cooldowns, and failure modes.

- This shows model governance in practice: models are resources with availability, cost, quality, quota, privacy, and failure characteristics.
- Cost-reduction claims should be treated as design goals unless fresh logs verify exact percentages.

What I had in mind

I did not want to treat model choice as taste. I wanted routing to be part of the architecture: cheap when possible, strong when necessary, local when privacy matters, and fallback-aware when a provider fails.

What I built

- `ai_router.py`, `cascade.py`, and `litellm_config.yaml` define model registries, routing strategies, tiering, quota tracking, fallback, race/cancel behavior, and budget-aware escalation.

Memory, Vault, CASS, and Smart Memory MCP

Long-running agents need more than a large context window. They need raw memory, compressed memory, procedural learning, contradiction handling, retrieval, and a way to avoid acting on stale facts.

- This shows that long-running agents need memory governance: raw records, summaries, procedural learning, contradictions, staleness, and retrieval metadata.
- Presented as practical memory infrastructure, not proof of human-like consciousness.

What I had in mind

The memory work came from noticing that agents do not only need more context. They need memory hygiene: raw facts, summaries, procedural learning, contradictions, staleness and provenance.

What I built

- `cass_pipeline.py`, `memory_broker.py`, vault-watcher design notes, and `smart-memory-mcp/server.js` show schemas, scoring, contradiction logic, TF-IDF retrieval, recency/usefulness metadata, and MCP tool declarations.

Voice Interface and Spoken Tool Use

Voice control is powerful because it lowers friction, but spoken commands are ambiguous and can trigger tools. The system needed tool routing, muting, fallbacks, destructive-command filtering, and a privacy-forward plan.

- This shows voice as a safety-sensitive command surface, not just a pleasant chat interface.
- Wake-word/macOS app features should be described as planned or partially integrated unless retested.

What I had in mind

Voice was exciting because it made the system feel immediate, but that is also why it is dangerous. If I can speak and the computer acts, then muting, confirmation, shell limits and privacy are part of the product, not accessories.

What I built

- `voice.py` contains Gemini Live config, sample-rate handling, tool declarations, dispatcher, vault tools, deep-think route, shell guard, reconnect/fallback loop, and VoiceBridge experiments. Planning notes define latency targets, privacy red lines, and acceptance criteria.